Optimizing Log Management for Microsoft Sentinel & Defender XDR

Best Practices

Deniz MUTLU

Hello & Welcome!

My name is Deniz Mutlu

Microsoft Security MVP & MCT

Work At : @ Swiss Post Cybersecurity (Hacknowledge)

Fancy Title : Director Strategic Partner Mgmt | Senior Security Engineer

https://linkedin.com/in/dmutlu

Agenda for today

- The Log Management problem
- 1) Log Management Basics
- 2) Sentinel & MXDR Deep Dive
- 3) Can we ingest everything like a “real” SIEM?
- Conclusion & Wrap-Up

The Core Problem: Bad Data → Bad Detection

- Optimizing Log Management for Microsoft Sentinel & Defender XDR why?
  - Maintainability over time
  - Compliance/audit
  - Powerful Hunting/investigation/Forensics SIEM platform
  - Cost controls
  - Etc.

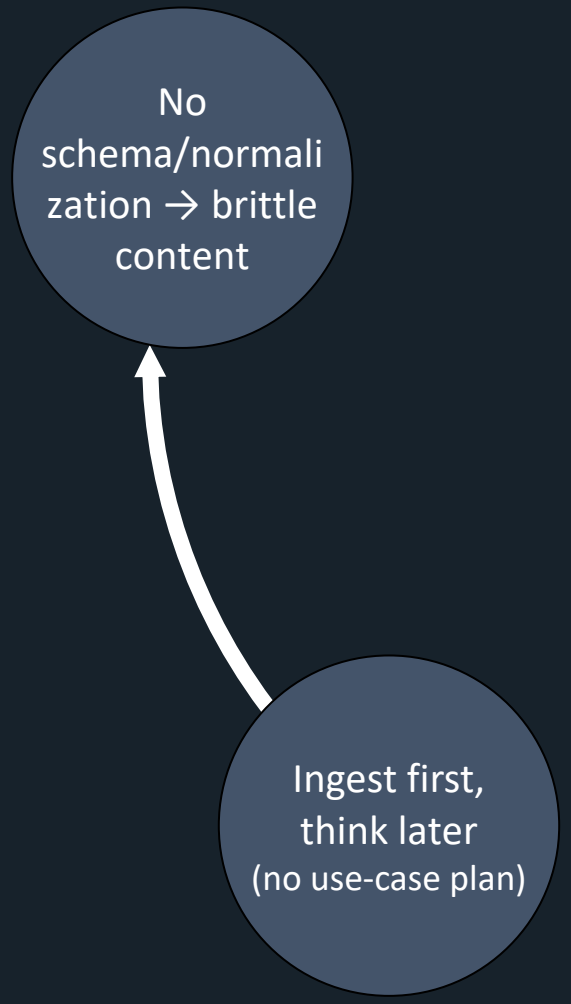
- Based on XP from +300 SOC customers and 70+ Sentinel/MXDR SOCs
- Live demo: ingesting anything (my chicken coop 🐔 → Sentinel)

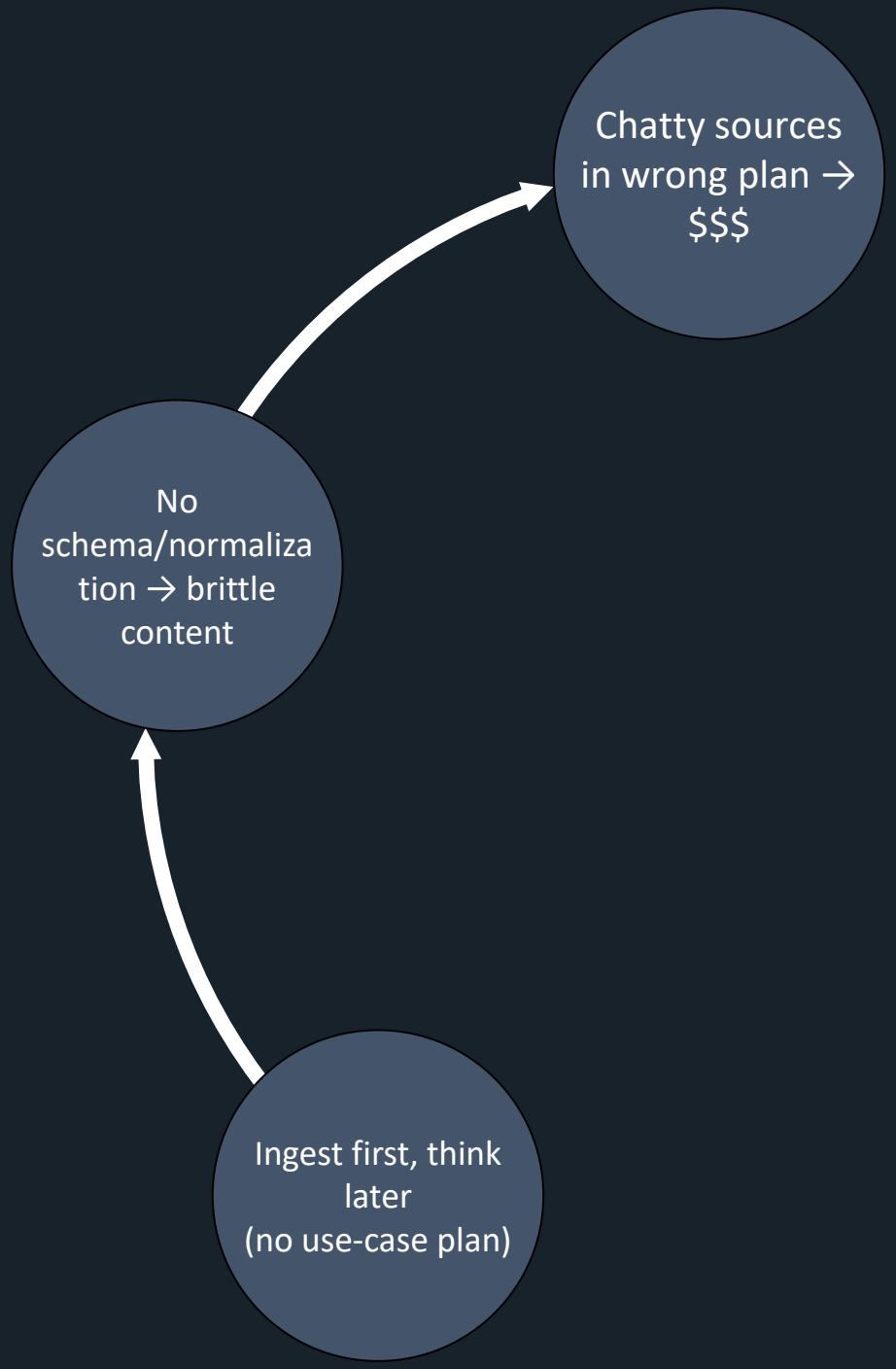
→Goal: better detections, cheaper ingestion, faster hunts

1) Log Management basics

The Core Problem: Bad Data → Bad Detection

Ingest first,
think later
(no use-case plan)


No schema/normalization → brittle content
Ingest first, think later
(no use-case plan)


Chatty sources in wrong plan → $$$
No schema/normalization → brittle content
Ingest first, think later (no use-case plan)


Chatty sources in wrong plan → $$$
Little/no parsing strategy → slow KQL
Ingest first, think later (no use-case plan)
No schema/normalization → brittle content


Chatty sources in wrong plan → $$$
Little/no parsing strategy → slow KQL
Retention chaos → gaps, audit pain
Ingest first, think later (no use-case plan)
No schema/normalization → brittle content
Why you are not happy


Chatty sources in wrong plan → $$$
Little/no parsing strategy → slow KQL
Retention chaos → gaps, audit pain
Ingest first, think later (no use-case plan)
No schema/normalization → brittle content
Why SOCs miss detections

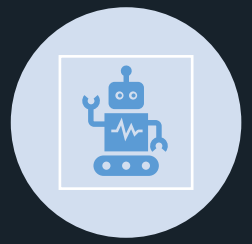
What we collect and why

- **What** we collect (by use case)
- **Why** (detection, hunt, audit, IR, regulation/law)
- **Who** owns each source/table (RACI)
- **How** it flows (transport, centralization, transforms)
- **Where** it lands (type, plan, workspace, lake)
- **How** long we keep it (interactive vs long-term)

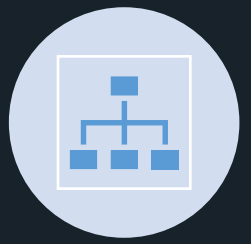
Mindset: Log Management Before SIEM Magic

- Logs ≠ SIEM → mid term SIEM success depends on log discipline
- Think quality, cost, governance before ingestion
- SOC maturity grows from policy/process → parsing → detection → Tools

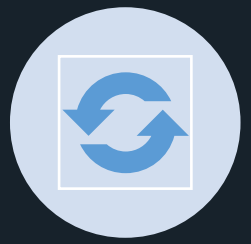
Roles & RACI



SOURCE OWNER: ONBOARD, HEALTH, UPDATES

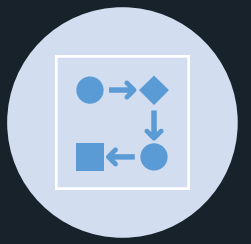


SEC ENG: SCHEMA, PARSERS, RULES, TUNING IR/



HUNTERS: REQUIREMENTS → FEEDBACK LOOP

PLATFORM: PLAN/RETENTION, COST, ACCESS



CISO/COMPLIANCE: RETENTION MANDATE, REVIEW CADENCE

Reference Architecture (3-tier)

Generate: M365, Azure, endpoints, SaaS, network, apps

Collect/Transform: AMA/DCR, CEF/Syslog, APIs, Functions

Store: LA (Analytics/Basic/Auxiliary), Sentinel Data Lake

Use: Analytics, Hunting, Workbooks, Automation, Correlation etc.

Good log vs bad log format

- A good log format must be composed in the following order with
  - A timestamp
  - The hostname
  - The process name
  - Valuable information returned by the process
    - A status, an action, an error, …

In a structured format
  - XML, key=value, JSON

Optionally, not verbose (for disk space reduction purposes)

A bad log is one not fitting one the above rules
  - But, bad logs are not necessary ignored
  - they are just harder to handle

Principal Log types

- Cloud Native Logs
- 3rd Party Logs (Data Connectors)
- Rest of the World
  - Raw files (human readable → ie. .log, json, syslog)
  - Binary files (ie. Winevents (.evl, evtx)
  - Databases (always a little nightmare…)

Azure & SaaS Sources

- Native connectors (Defender, Azure AD, O365) → resource-specific tables
- SaaS apps: Graph API, vendor APIs
- Watch for table-level plan choices (Analytics vs Basic)
- Best practice: test in staging → validate schema → move to prod

Collection Patterns: Windows & *Nix

- Windows: WEF/WEC, AMA for event logs
- Linux/Network: Syslog/CEF
- 3rd-party SaaS: APIs, custom connectors
- Custom logs: AMA text log collection, Logs Ingestion API, DB

Example of bad logs (Windows)

+ System
- EventData

MemberName	cn=adm-dmu,ou=WAD Ad
MemberSid	S-1-5-21-2457413560-29
TargetUserName	Protected Users
TargetDomainName	WAD
TargetSid	S-1-5-21-2457413560-29
SubjectUserSid	S-1-5-21-2457413560-29
SubjectUserName	Administrator
SubjectDomainName	WAD
SubjectLogonId	0x660e6
PrivilegeList	-

+ System
- EventData

SubjectUserSid	S-1-5-18
SubjectUserName	CHLPWDC011$
SubjectDomainName	BIN
SubjectLogonId	0x3e7
TargetUserSid	S-1-5-21-1821916493-3544644797-4237041638-1134
TargetUserName	svc_splunk
TargetDomainName	BIN
TargetLogonId	0xb4a1712
LogonType	3
LogonProcessName	Advapi
AuthenticationPackageName	MICROSOFT_AUTHENTICATION_PACKAGE_V1_0
WorkstationName	CHLPWDC011
LogonGuid	{00000000-0000-0000-0000-000000000000}
TransmittedServices	-
LmPackageName	-
KeyLength	0
ProcessId	0x1e0
ProcessName	C:\Windows\System32\lsass.exe
IpAddress	172.20.4.10
IpPort	55470
ImpersonationLevel	%%1833

+ System
- EventData

TargetUserSid	S-1-5-18
TargetUserName	WAD-DC-SRV2$
TargetDomainName	WAD
TargetLogonId	0x13bdc7
LogonType	3

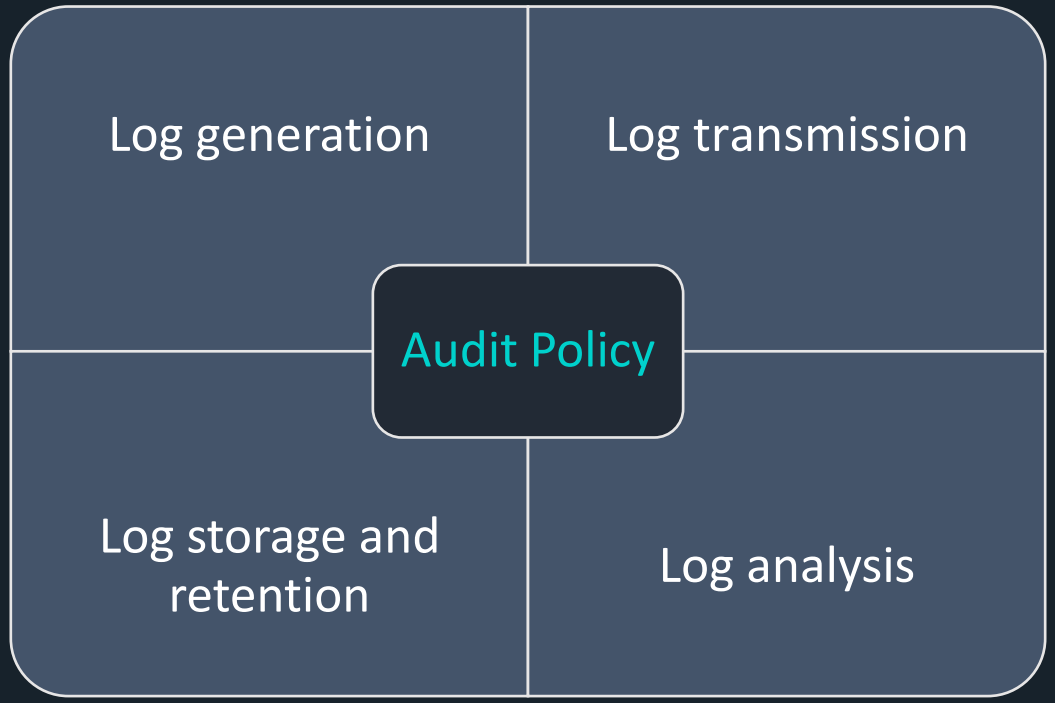
Retention concepts: Hot / Warm / Cold Data

- Hot (interactive): 30–90d → active detections/hunting
- Warm (extended): 6–24m → compliance, advanced hunts
- Cold (archive): 1–7y → forensics, audit
- Freeze (Archive) 1–10y → audit, compliance

Always: document owner + retention reason

Defining a strong audit policy

- 4 aspects needs to be covered for the definition of the audit policy



Roles & responsibilities?

Log transmission (not Cloud)

Collection Sensor

Magic Tool

Logstash
Nxlog
Winlogbeat
Cribl
Tenzir
Etc.

**Data Connectors Machines*

Log Tuning
Drop fields
Normalization
Anonymization
Etc.

Log optimization

Logs Source

Logs

Send

Send

Sentinel

Reduce volumetry
|
Reduce costs
|
Improve data quality

🧑‍🤝‍🧑 Roles & Responsibilities (RACI)

- **Responsible:** SOC engineers / log owners (configure, forward logs)
- **Accountable:** CISO / Security leadership (approve log policy, retention)
- **Consulted:** IT Ops, Compliance, App owners (requirements, exceptions)
- **Informed:** Auditors, Management (reports, assurance)

🏛️ Reference Architecture

- **Source** → Cloud (Azure, M365, SaaS), On-prem (FW, EDR, Syslog), Custom apps
- **Transport** → Native connector, Event Hub, AMA/agent, API push (Logs Ingestion)
- **Ingestion** → Sentinel (Analytics/Basic/Aux logs), Data Lake, Archive (S3/Blob)
- **Storage tiers:**
  - **Hot (0–90d):** high-fidelity, detection-ready logs
  - **Warm (3–12m):** less frequently queried, compressed
  - **Cold (1y+):** regulatory/archive, query-on-demand

📝 Policy Checklist

- [ ] Define **scope**: which systems, apps, services must send logs
- [ ] Define **log categories:** Security, Compliance, Operational, Audit
- [ ] Define **ownership**: who is responsible per log source (RACI)
- [ ] Define **quality standards:** time sync (NTP), UTF-8, JSON, include User/IP/Action
- [ ] Define **normalization rules:** field naming (TimeGenerated, DeviceId, EventType, etc.)
- [ ] Define **retention** per category (90d, 180d, 1y, 7y)
- [ ] Define **storage tiers:** Hot vs Warm vs Cold
- [ ] Define **cost control measures:** filtering, splitting tables, compression
- [ ] Define **alerting coverage:** map MITRE ATT&CK / detection gaps
- [ ] Define **audit requirements:** FINMA, CSSF, SOX, PCI, ISO 27001, NIS2, etc.
- [ ] Define **review cadence:** log policy reviewed quarterly/annually (assign owner)

💡 Best Practices

- Enable **time sync (NTP)** everywhere → prevents correlation errors.
- Always map logs to **TimeGenerated** in Sentinel.
- Use **splitting techniques** (analytics vs basic vs auxiliary logs)

Log Management Cheat Sheet

- KQL-labs-parsing/ChickenCoop/LogManagement_CheatSheet.md at main · DenMutlu/KQL-labs-parsing

- Scope / Ownership / Retention
- Normalize / Filter / Cost control
- Hot vs Warm vs Cold
- Review policy regularly

Bad logs = bad detections. Good policy = effective SOC.

2) Sentinel & MXDR Deep Dive

Now let’s dive into Sentinel & Azure-specific practices.

Sentinel Log Plans Overview

- Analytics: multi-table, detection-grade, high performance
- Basic: single-table, troubleshooting, dashboards
- Auxiliary (PP): single-table, low-fidelity ops/audit

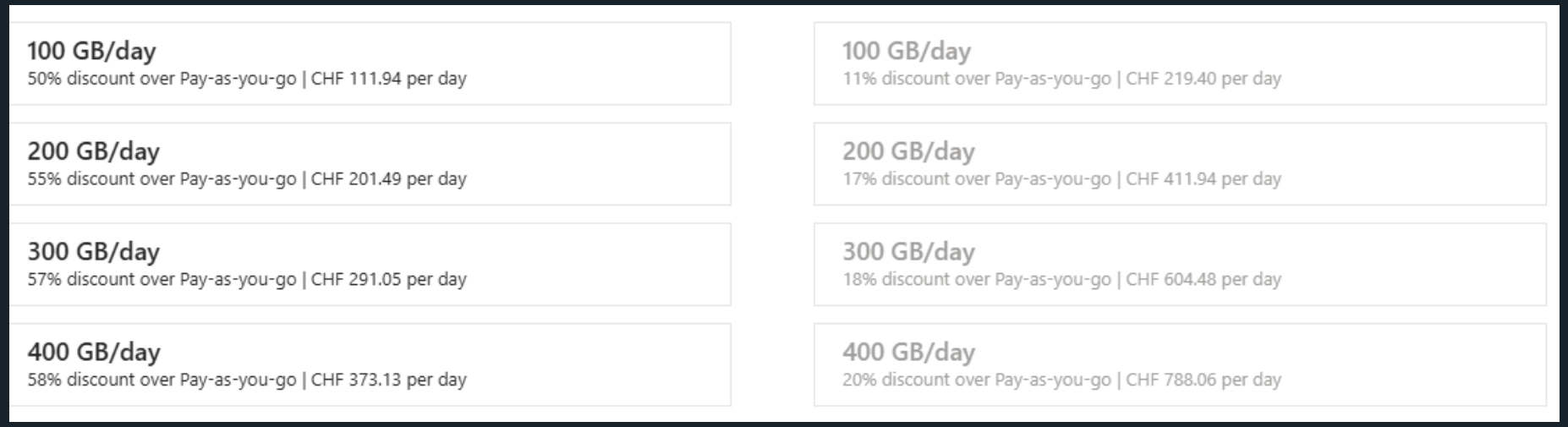
Retention strategy that scales

- Analytics: 30–90d hot (interactive)
- Basic/Aux: 30d interactive (cheap)
- Data Lake or external: 1–10y cold (KQL Jobs/Search Jobs)

Keep hot only what detection needs; push the rest cold.

Costs Strategy

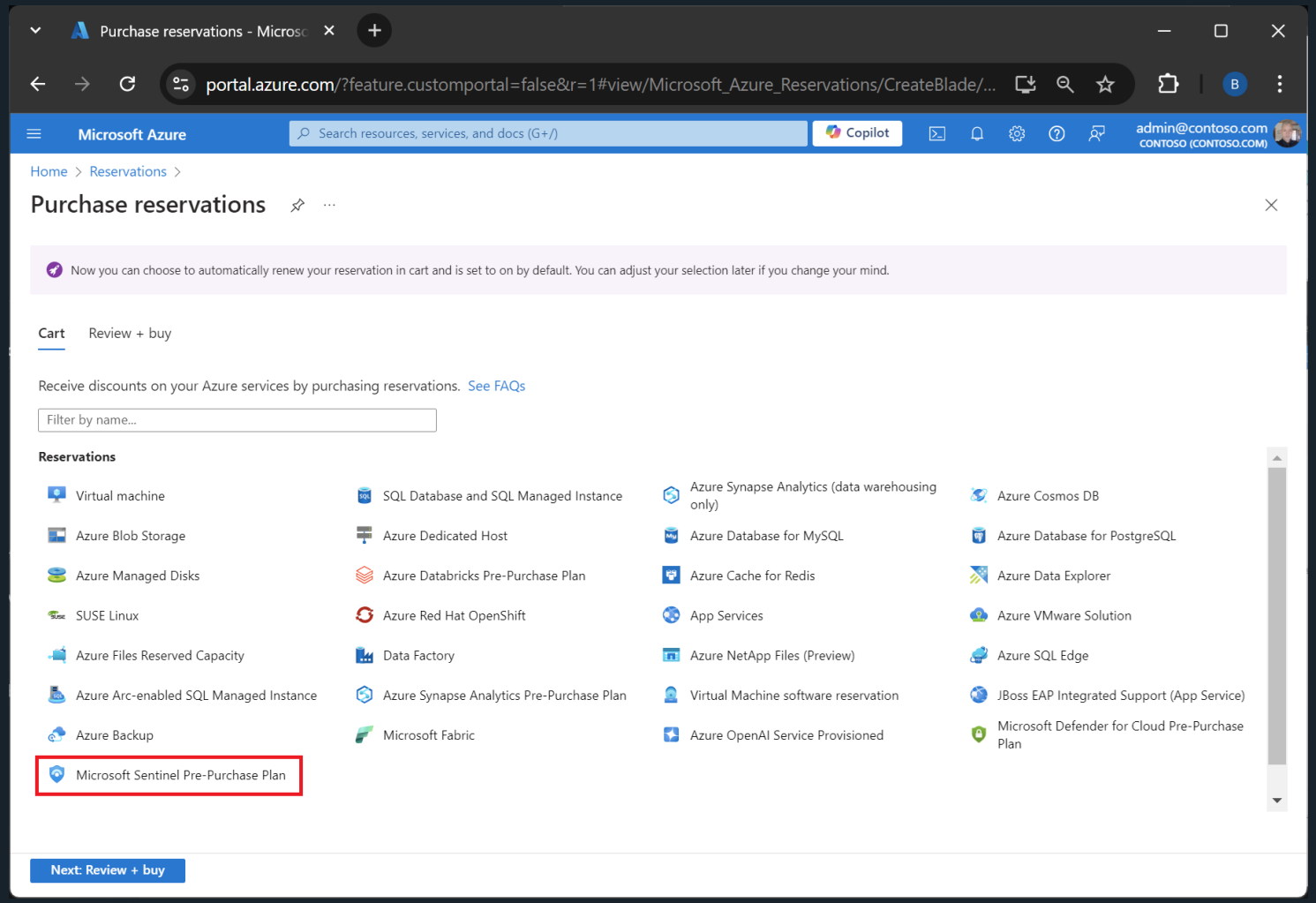
- Pay-as-you Go Model
- **Microsoft Sentinel pricing / Log Analytics pricing**



Microsoft Sentinel - Savings

Microsoft 365 E5 Benefit	Customers on E5 suite receive a grant of **5 MB per user/day** to ingest Microsoft 365 data	Microsoft 365 E5 benefit offer with Microsoft Sentinel | Microsoft Azure
Defender for Server P2 Benefit	**500 MB per VM per day** of free data ingestion in both Sentinel and Log Analytics. The allowance is specifically for the security data types that are directly collected by Defender	Common questions - Defender for Servers - Microsoft Defender for Cloud | Microsoft Learn
Free data sources	Some Azure tables are free from data ingestion charges altogether	Plan costs and understand pricing and billing - Microsoft Sentinel | Microsoft Learn
Seasonality impact	Ingestion volume normally drops during weekends (**~32%** on average)	
P3 Savings Plan	Prepaid annual commitment apply to all Microsoft Sentinel pricing tiers, excluding Azure Monitor tiers, Data Retention, Restore and Search	

Purchase Microsoft Sentinel commit units



Ingestion Pipe

Analytics Logs

Ingestion cost
Standard

Powerful all inclusive logs ideal for real-time monitoring, alerting and dashboards. Including unlimited queries30 days interactive retention included – can extend to 2y, up to 12y

Basic Logs

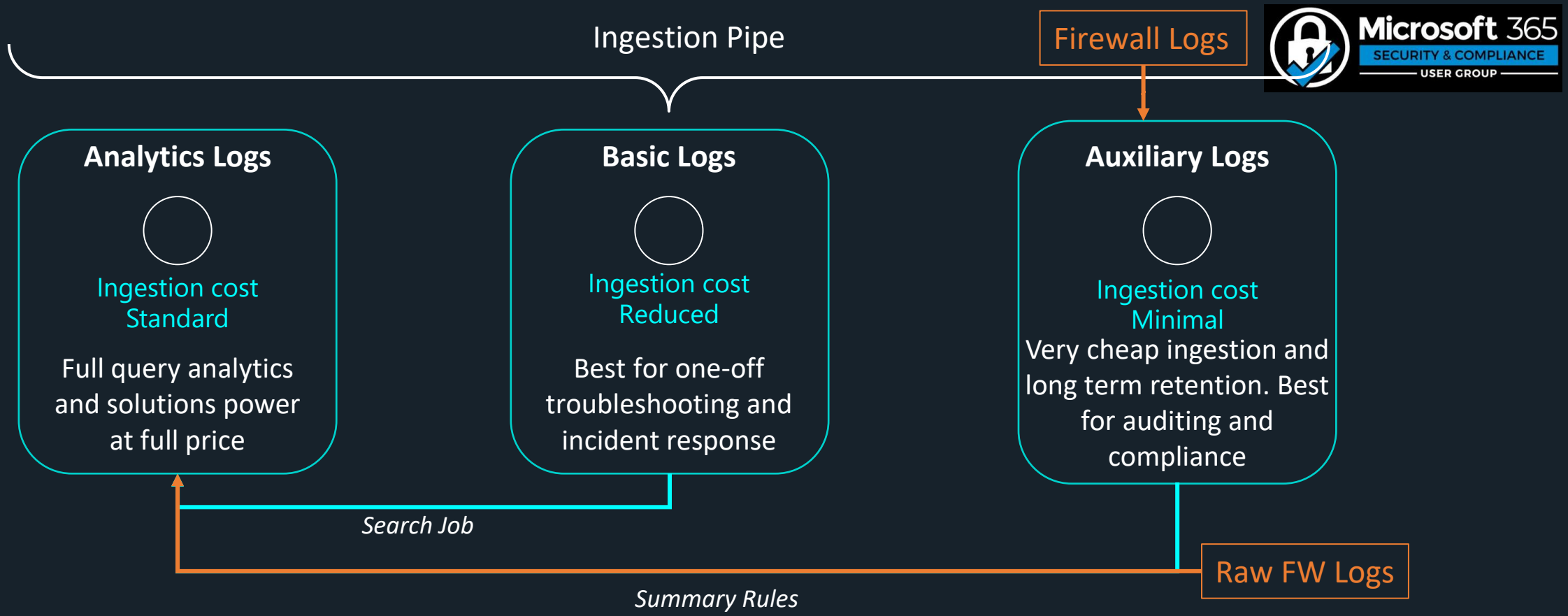
Ingestion cost
Reduced

Medium-touch telemetry data needed for troubleshooting and incident response.
Additional charges for queries
30 days interactive retention
Long Term retention up to 12y

Auxiliary Logs

Ingestion cost
Minimal

Low-touch telemetry data intended for high verbose logs, auditing and compliance.
Additional charges for queries
30 days interactive retention
Long Term retention up to 12y


Ingestion Pipe
Firewall Logs
Analytics Logs
Ingestion cost
Standard
Full query analytics and solutions power at full price
Basic Logs
Ingestion cost
Reduced
Best for one-off troubleshooting and incident response
Auxiliary Logs
Ingestion cost
Minimal
Very cheap ingestion and long term retention. Best for auditing and compliance
Search Job
Raw FW Logs
Summary Rules

Auxiliary Logs

- New log type (30-day interactive only)
- Optimized for audit/ops events (low-value security)
- Cheapest option for noisy but necessary logs
- No cross-table joins → plan carefully

Summary rules

- Convert high-rate logs → 1 row / 5–15 min

Example:

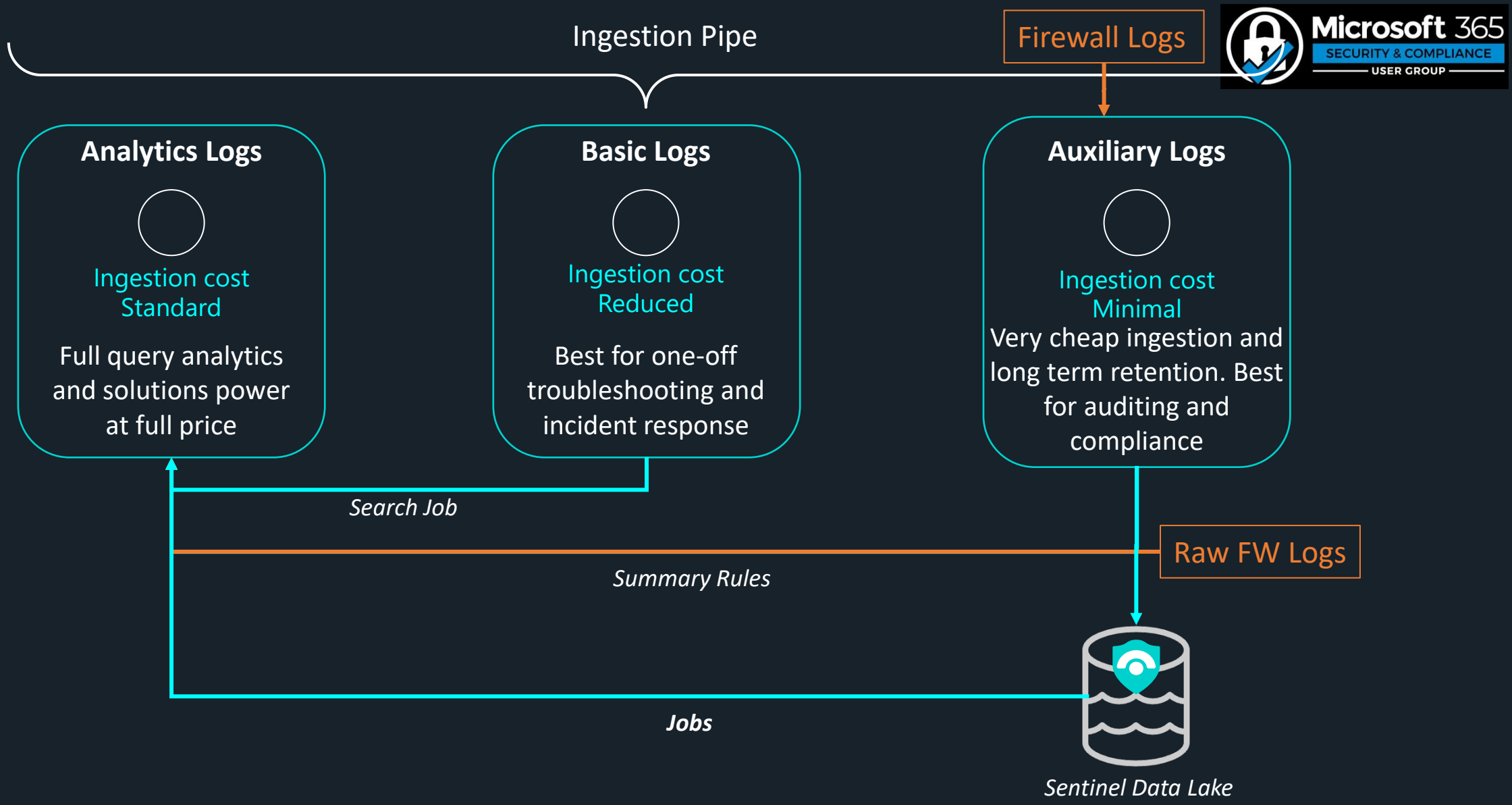
Source_CL

```
| summarize
    Count = count(),
    DistinctUsers = dcount(User),
    MaxTemp = max(TempC)
  by bin(TimeGenerated, 5m), DeviceId
```

- Use summaries for dashboards & triage
- Keep raw only where detection requires it

Sentinel Data Lake (preview): why it matters

- Decoupled storage/compute; query with KQL
- Long-term, low-cost log retention
- Great for compliance, forensics, historical hunting
- Pattern: Hot (Analytics) + Cold (Lake)

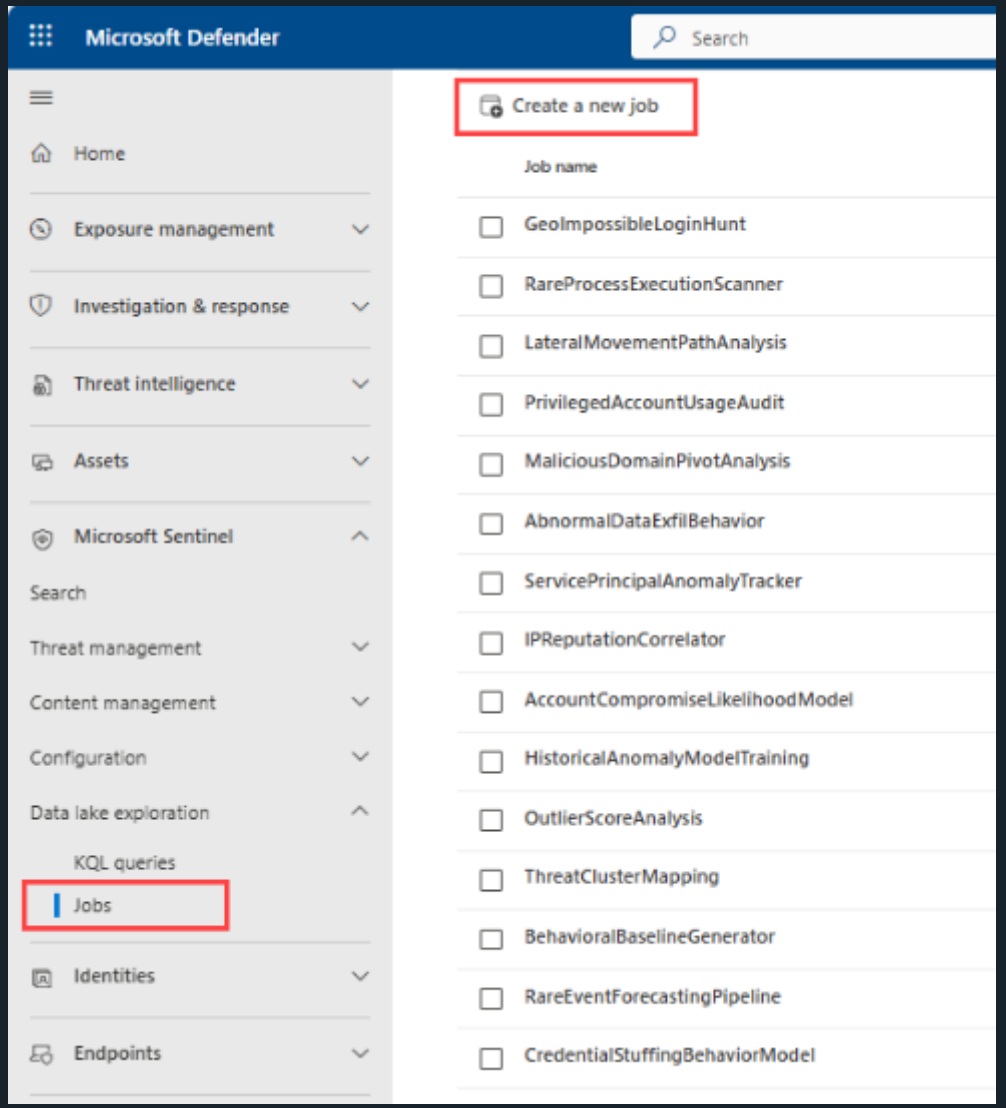

Ingestion Pipe
Firewall Logs
Analytics Logs
Ingestion cost
Standard
Full query analytics and solutions power at full price
Basic Logs
Ingestion cost
Reduced
Best for one-off troubleshooting and incident response
Auxiliary Logs
Ingestion cost
Minimal
Very cheap ingestion and long term retention. Best for auditing and compliance
Search Job
Summary Rules
Raw FW Logs
Jobs
Sentinel Data Lake

Sentinel Data Lake Jobs – Hidden Power

- Run scheduled KQL queries on archived data in Data Lake
- Store results in dedicated tables for cheap detection
- Use results as input for Analytics rules & hunting

Examples:

  - Daily user anomaly summary
  - Weekly impossible travel pre-aggregation
  - Massive volume parsing offloaded from Analytics


Microsoft Defender
Search
Create a new job
Home
Exposure management
Investigation & response
Threat intelligence
Assets
Microsoft Sentinel
Search
Threat management
Content management
Configuration
Data lake exploration
KQL queries
Jobs
Identities
Endpoints
Job name
GeoImpossibleLoginHunt
RareProcessExecutionScanner
LateralMovementPathAnalysis
PrivilegedAccountUsageAudit
MaliciousDomainPivotAnalysis
AbnormalDataExfilBehavior
ServicePrincipalAnomalyTracker
IPReputationCorrelator
AccountCompromiseLikelihoodModel
HistoricalAnomalyModelTraining
OutlierScoreAnalysis
ThreatClusterMapping
BehavioralBaselineGenerator
RareEventForecastingPipeline
CredentialStuffingBehaviorModel

Create a new KQL job

- Name & details
- **Query**
- Schedule
- Summary

Review the query

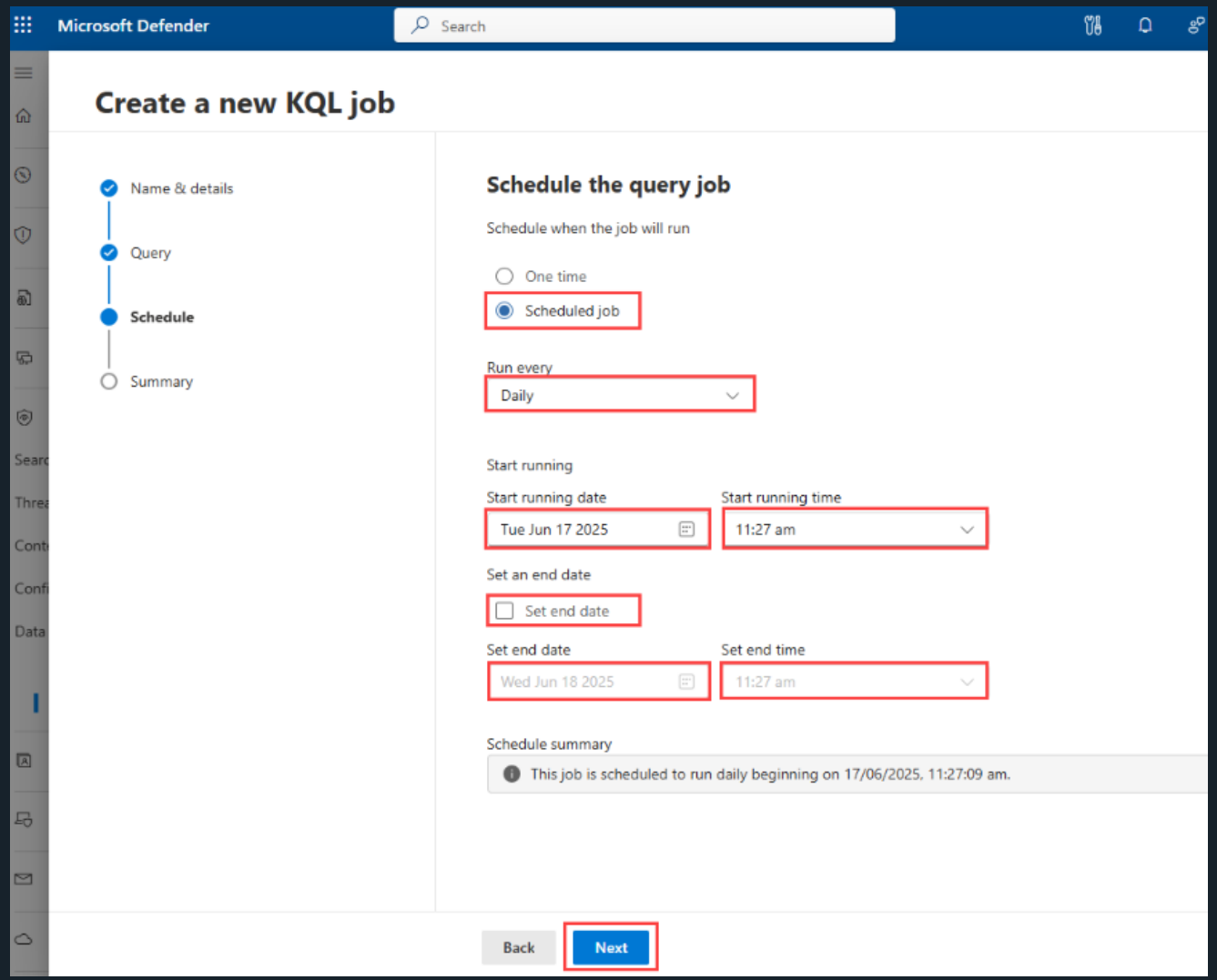
Review and edit the query to make sure it has what you need.

Last 24 hours

Selected workspace: Contoso-LogAnalyiticsWorkspace

```
let StartDate = ago(180d);
let EndDate = now();
let BinTime = 1h;
let Sensitivity = 3.0;
SigninLogs
| where TimeGenerated between (StartDate .. EndDate)
| where isnotempty(UserPrincipalName)
| where ResultType != 0  // Focus on failed or suspicious sign-ins
| summarize TimeSeries = make_list(pack("Time", TimeGenerated, "Count", 1)) by Us
| mv-expand TimeSeries
| evaluate bag_unpack(TimeSeries)
| project UserPrincipalName, TimeGenerated = todatetime(Time), Count = toint(Coun
| summarize CountSeries = make_list(Count), TimeSeries = make_list(TimeGenerated)
| extend (Anomalies, Score, Baseline) = series_decompose_anomalies(CountSeries, S
| mv-expand TimeGenerated = TimeSeries, Count = CountSeries, Anomalies, Score, Ba
| where Anomalies > 0 and Baseline > 0
```

Back | Next | Cancel

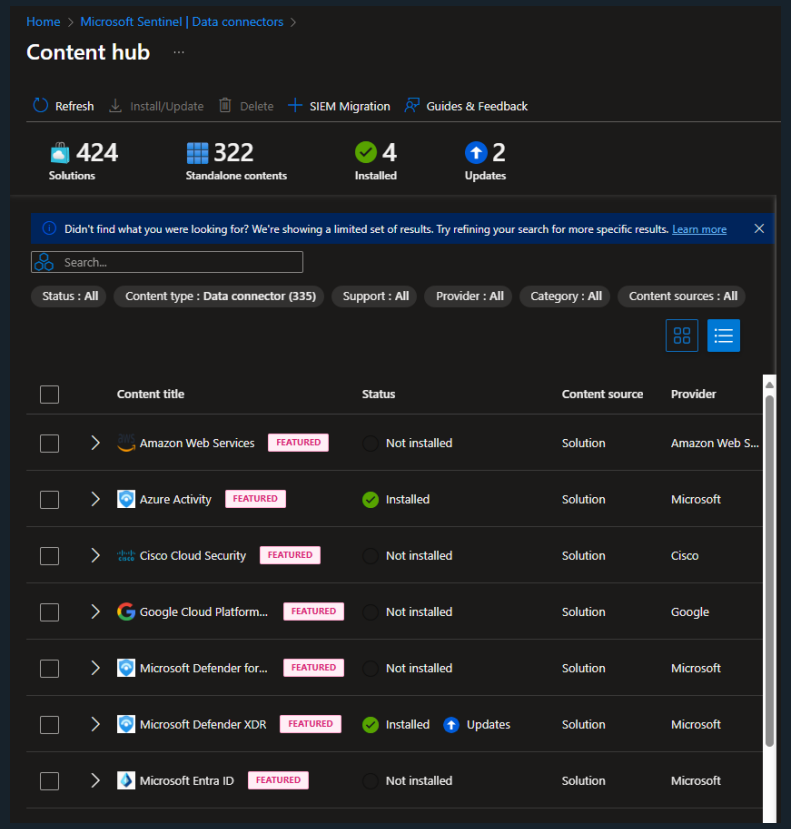

Microsoft Defender
Search
Create a new KQL job
Name & details
Query
Schedule
Summary
Schedule the query job
Schedule when the job will run
One time
Scheduled job
Run every
Daily
Start running
Start running date
Tue Jun 17 2025
Start running time
11:27 am
Set an end date
Set end date
Set end date
Wed Jun 18 2025
Set end time
11:27 am
Schedule summary
This job is scheduled to run daily beginning on 17/06/2025, 11:27:09 am.
Back
Next

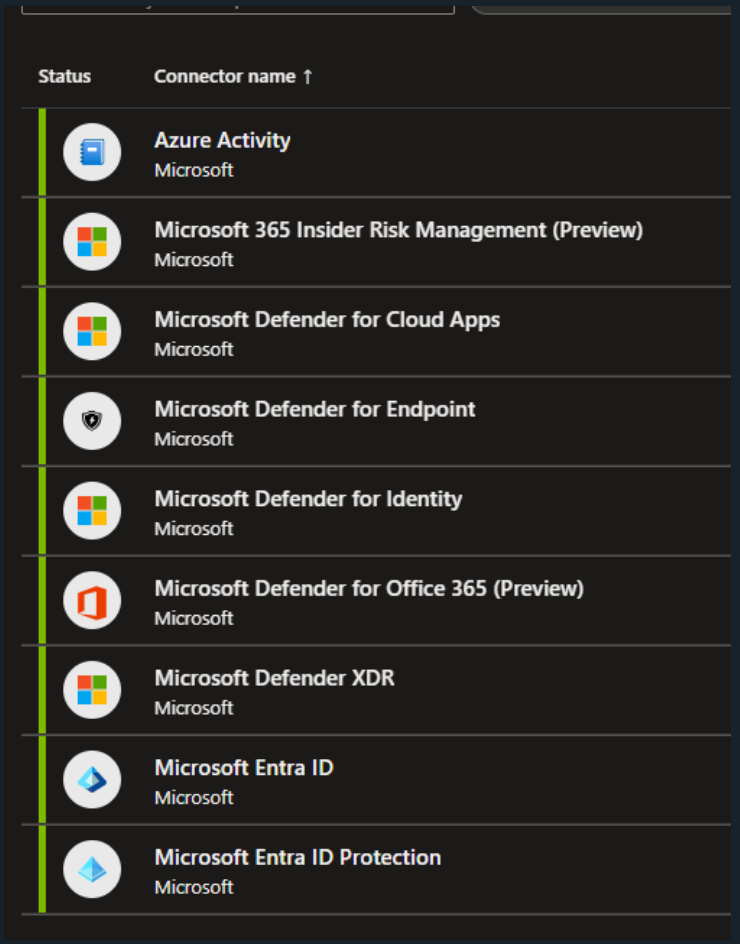
Data quality: make logs “analysis-ready”

- Mandatory fields: Time, Device/Host, User/Account, Action/Outcome
- Table names (normalize/alias); avoid free-text for key values
- Prefer JSON over ad-hoc text; add Vendor, Product, EventType
- GUID/IDs over display names; include TenantId / SourceIP

Ingestion methods

- First-party: M365/Azure native connectors from the Content Hub, resource-specific tables





Free Data Sources

- Azure Activity Logs
- Microsoft Sentinel Health
- Office 365 Audit Logs, including all SharePoint activity, Exchange admin activity, and Teams
- Security alerts, including alerts from the following sources:
- Microsoft Defender XDR
- Microsoft Defender for Cloud
- Microsoft Defender for Office 365
- Microsoft Defender for Identity
- Microsoft Defender for Cloud Apps
- Microsoft Defender for Endpoint
- Alerts from the following sources:
- Microsoft Defender for Cloud
- Microsoft Defender for Cloud Apps
- Plan costs and understand pricing and billing - Microsoft Sentinel | Microsoft Learn

Ingestion methods

3rd-party:

- Azure Monitor agent (AMA) logs, such as:
- Windows Security Events via AMA
- Windows Forwarded Events
- CEF data
- Syslog data
- Direct ingestion via Logs ingestion API
- Built-in, API-based data connector, such as:
  - Codeless data connectors

Transform early: DCR mappings, drop noise, normalize names

Governance Must-Haves

- Availability: buffering, backfill, monitoring
- Integrity: immutability, chain of custody
- Confidentiality: PII tagging, RBAC, masking
- time sync (NTP) everywhere
- Control Access: least privilege in Sentinel & Lake

3) But can Sentinel ingest anything?

Chicken Coop Demo

Do you hear what I hear?

- Sentinel is not a real SIEM
- We can not ingest anything
- Data not directly from cloud are complex to handle in Sentinel
- Bla bla bla…

- Let’s try, but before..

Ingestion methods (what to choose when)

Pull (Connectors):

Built-in Sentinel connectors (M365, Defender, Entra, AWS, etc.).

Agent-based (AMA custom text logs) or event streaming (Event Hub).

Pros: supported, easy lifecycle; Cons: fixed schema, may not exist for niche sources.

Push (APIs):

Logs Ingestion API (modern, DCR-based; what i used for this example).

Legacy HTTP Data Collector (avoid for new).

Pros: ingest **anything**, full control; Cons: you own sender, duplicates risk, throttling.

Other patterns:

Storage → Logic App/Function → Logs Ingestion API.

3rd-party forwarders → Event Hub → Data Collector.

Rule of thumb:

Use **built-in connectors** when available and use **Logs Ingestion API** for custom/long tail.

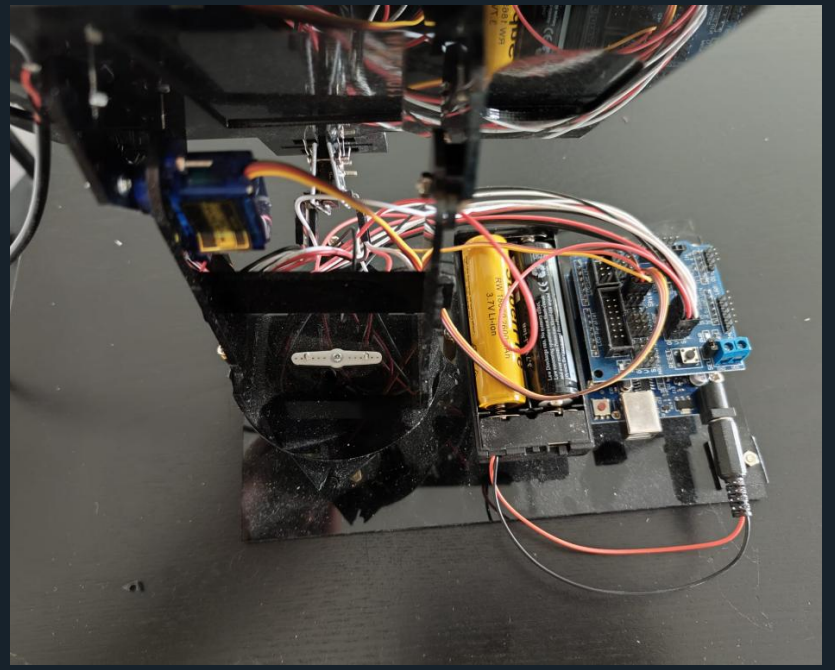
Demo - My Chicken Coop SOC!

- I have 8 chickens in my garden 🥚🥚🥚🥚🥚🥚
- I don’t want to spend too much time on daily maintenance
- → Automation is the key for everything ☺

- *Automatic door with light sensor (Raspberry Pi & Uno Mini)*
- *Water control system (Arduino: solar panel, battery, heater, temp sensor)*
- *Lighting controlled by ESP → Zigbee*
- *Automatic DIY feeder (no logs yet)*
- *Camera for fox monitoring (detection logs coming in Chicken Coop v4)*

Chicken Coop Automation V2

Chicken Coop Automation V2



2 raw logs generated

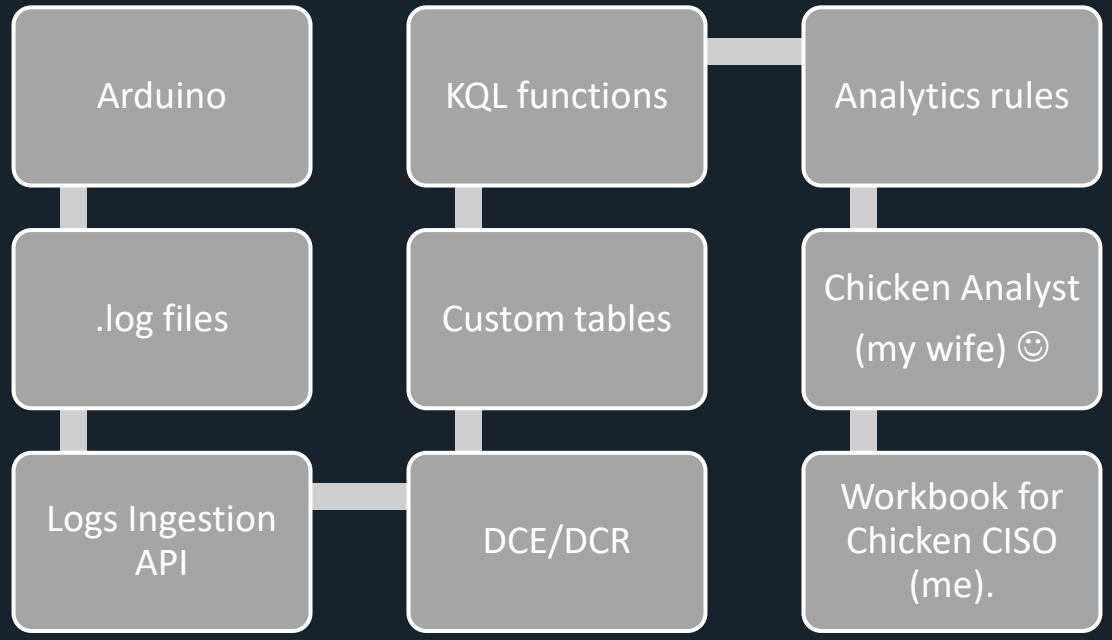
- Water Heater Logs → ChickenCoopDoor_CL


```
false, "source": "arduino-water", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Hourly water temp reading (summer)"}
{"time": "2025-08-01T09:00:00Z", "localTime": "2025-08-01T11:00:00+02:00", "deviceId": "coop-01", "eventType": "WaterTemp", "tempC": 17.39, "heaterOn": false, "batteryLow":
false, "source": "arduino-water", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Hourly water temp reading (summer)"}
{"time": "2025-08-01T10:00:00Z", "localTime": "2025-08-01T12:00:00+02:00", "deviceId": "coop-01", "eventType": "WaterTemp", "tempC": 17.56, "heaterOn": false, "batteryLow":
false, "source": "arduino-water", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Hourly water temp reading (summer)"}
{"time": "2025-08-01T11:00:00Z", "localTime": "2025-08-01T13:00:00+02:00", "deviceId": "coop-01", "eventType": "WaterTemp", "tempC": 17.7, "heaterOn": false, "batteryLow":
false, "source": "arduino-water", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Hourly water temp reading (summer)"}
{"time": "2025-08-01T12:00:00Z", "localTime": "2025-08-01T14:00:00+02:00", "deviceId": "coop-01", "eventType": "WaterTemp", "tempC": 18.96, "heaterOn": false, "batteryLow":
false, "source": "arduino-water", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Hourly water temp reading (summer)"}
{"time": "2025-08-01T13:00:00Z", "localTime": "2025-08-01T15:00:00+02:00", "deviceId": "coop-01", "eventType": "WaterTemp", "tempC": 18.41, "heaterOn": false, "batteryLow":
false, "source": "arduino-water", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Hourly water temp reading (summer)"}
```

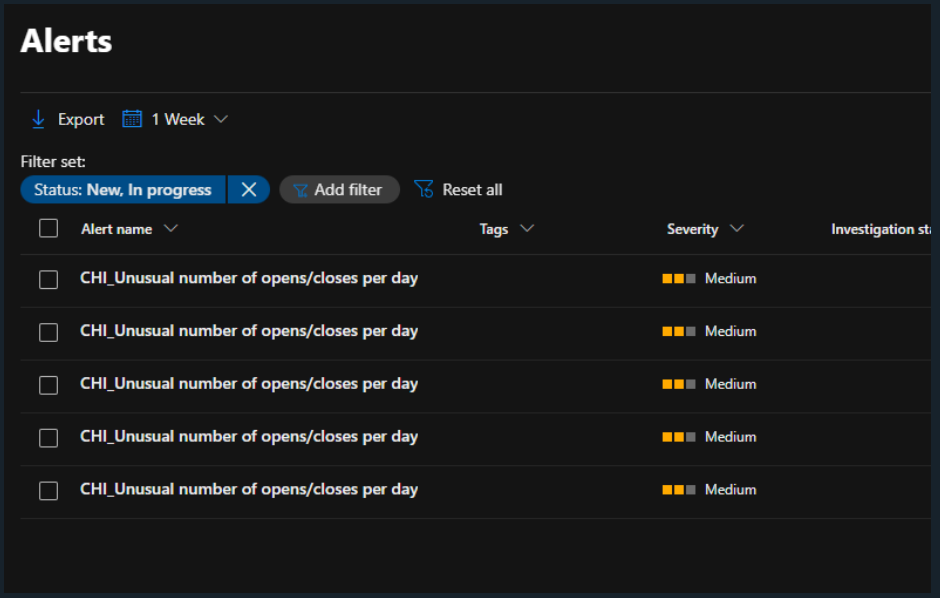

- Door Sensor → ChickenCoopWater_CL

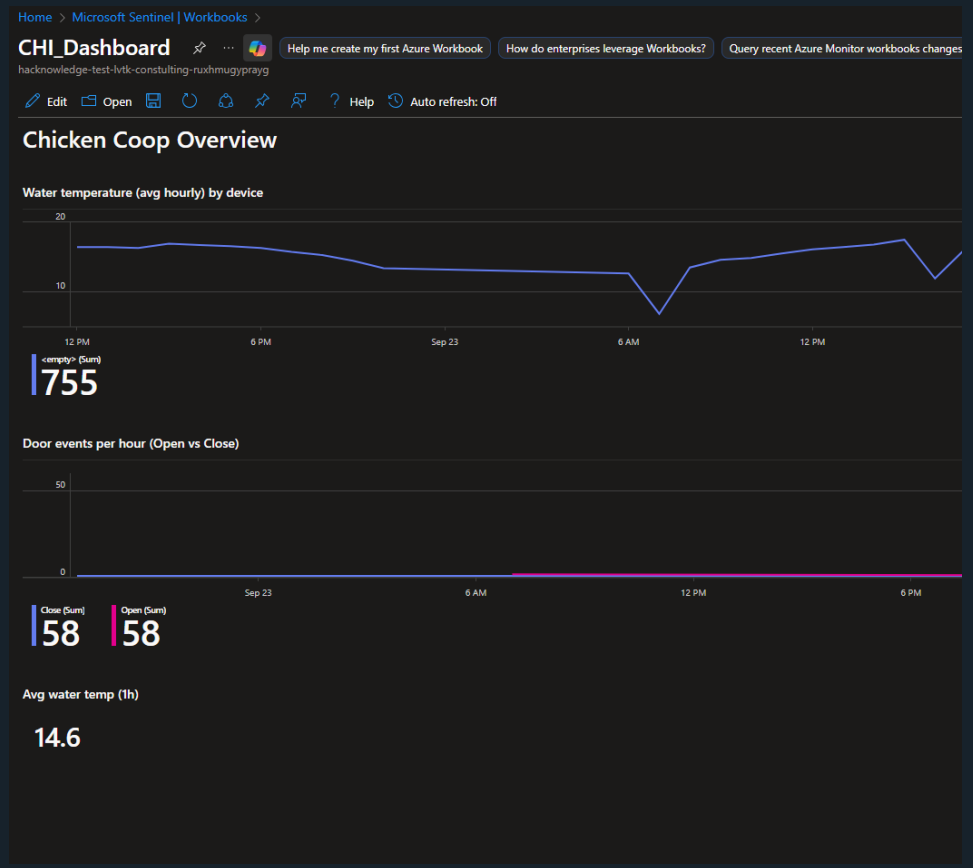

```
"solarLux": 31584, "source": "arduino-door", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Door opened automatically at sunrise"}
{"time": "2025-08-01T18:59:07Z", "localTime": "2025-08-01T20:59:07+02:00", "deviceId": "coop-01", "eventType": "Door", "doorState": "Close", "method": "LightSensor",
"solarLux": 206, "source": "arduino-door", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Door closed automatically at sunset"}
{"time": "2025-08-02T04:18:25Z", "localTime": "2025-08-02T06:18:25+02:00", "deviceId": "coop-01", "eventType": "Door", "doorState": "Open", "method": "LightSensor",
"solarLux": 27238, "source": "arduino-door", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Door opened automatically at sunrise"}
{"time": "2025-08-02T19:01:47Z", "localTime": "2025-08-02T21:01:47+02:00", "deviceId": "coop-01", "eventType": "Door", "doorState": "Close", "method": "LightSensor",
"solarLux": 163, "source": "arduino-door", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Door closed automatically at sunset"}
{"time": "2025-08-03T04:16:38Z", "localTime": "2025-08-03T06:16:38+02:00", "deviceId": "coop-01", "eventType": "Door", "doorState": "Open", "method": "LightSensor",
"solarLux": 28483, "source": "arduino-door", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Door opened automatically at sunrise"}
{"time": "2025-08-03T19:05:25Z", "localTime": "2025-08-03T21:05:25+02:00", "deviceId": "coop-01", "eventType": "Door", "doorState": "Close", "method": "LightSensor",
"solarLux": 210, "source": "arduino-door", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Door closed automatically at sunset"}
{"time": "2025-08-04T04:13:50Z", "localTime": "2025-08-04T06:13:50+02:00", "deviceId": "coop-01", "eventType": "Door", "doorState": "Open", "method": "LightSensor",
"solarLux": 33449, "source": "arduino-door", "location": "Monthey,CH", "latitude": 46.25, "longitude": 6.95, "message": "Door opened automatically at sunrise"}
{"time": "2025-08-04T19:02:02Z", "localTime": "2025-08-04T21:02:02+02:00", "deviceId": "coop-01", "eventType": "Door", "doorState": "Close", "method": "LightSensor",
```

Demo architecture Chicken Coop SOC



Quick Demo


Alerts
Export
1 Week
Filter set:
Status: New, In progress
Add filter
Reset all
Alert name
Tags
Severity
Investigation st
CHI_Unusual number of opens/closes per day
Medium
CHI_Unusual number of opens/closes per day
Medium
CHI_Unusual number of opens/closes per day
Medium
CHI_Unusual number of opens/closes per day
Medium
CHI_Unusual number of opens/closes per day
Medium


Home > Microsoft Sentinel | Workbooks >
CHI_Dashboard
Help me create my first Azure Workbook
How do enterprises leverage Workbooks?
Query recent Azure Monitor workbooks changes
hacknowledge-test-lvtk-constulting-ruxhmugyprayg
Edit
Open
Help
Auto refresh: Off
Chicken Coop Overview
Water temperature (avg hourly) by device
20
10
12 PM
6 PM
Sep 23
6 AM
12 PM
<empty> (Sum)
755
Door events per hour (Open vs Close)
50
0
Sep 23
6 AM
12 PM
6 PM
Close (Sum)
58
Open (Sum)
58
Avg water temp (1h)
14.6

Demo concept: “Chicken Coop → Sentinel”

- KQL-labs-parsing/ChickenCoop/Sentinel_ingestion.md at main · DenMutlu/KQL-labs-parsing

Conclusion & Wrap-Up

Common Pitfalls & Quick Fixes

Pitfalls:

- Ingesting everything → huge costs, little valued
- Duplicate data → wasted money + false detections
- Wrong retention → either missing data or bloated costs
- Schema drift → queries & rules break silently
- Ignoring roles → no one accountable when logs stop flowing

Quick Fixes:

- Define scope & ownership (RACI) early
- Apply log plans (Analytics / Basic / Auxiliary)
- Always normalize → TimeGenerated, DeviceId, EventType etc.
- Monitor ingestion cost trends & adjust
- Automate with DCRs and transformations

Wrap-Up & Key Messages

- Effective log management = foundation for Sentinel & XDR
- Data quality is a security control
- Cost optimization is about strategy, not cutting visibility
- Use retention tiers + Data Lake Jobs for scale
- Sentinel can ingest anything (API, connectors, custom sources)

Conclusion

- If we can SOC a chicken coop, we can SOC anything 🐔 ➡️ 🔐
- *Data born in the cloud stays in the cloud* → collect, manage, secure where it lives
- A SOC is not about tools, it’s about policy, process, and data quality
- Start small, prove value, then expand log coverage smartly

Thank You

To the team behind the

Microsoft 365 Security & Compliance User Group

The Sponsors

Threatscape

Packt

ENOW

And the Community !

Takeaways & Resources

- Azure Monitor Logs - Azure Monitor | Microsoft Learn
- When to use the Microsoft Sentinel data lake | Microsoft Learn
- Log retention plans in Microsoft Sentinel | Microsoft Learn
- Microsoft Sentinel Blog | Microsoft Community Hub
- Microsoft Sentinel data lake overview (preview) - Microsoft Security | Microsoft Learn
- Create jobs in the Microsoft Sentinel data lake (preview) - Microsoft Security | Microsoft Learn
- Manage KQL jobs - Microsoft Security | Microsoft Learn
- Run KQL queries against the Microsoft Sentinel data lake (preview) - Microsoft Security | Microsoft Learn
- Search for specific events across large datasets in Microsoft Sentinel | Microsoft Learn
- Create data collection rules (DCRs) in Azure Monitor - Azure Monitor | Microsoft Learn
- Data collection rules in Azure Monitor - Azure Monitor | Microsoft Learn
- Logs Ingestion API in Azure Monitor - Azure Monitor | Microsoft Learn
- Set up a table with the Auxiliary plan for low-cost data ingestion and retention in your Log Analytics workspace - Azure Monitor | Microsoft Learn
- KQL-labs-parsing/ChickenCoop/Sentinel_ingestion.md at main · DenMutlu/KQL-labs-parsing
- September 2025 - M365 Security & Compliance User Group, Wed, Sep 24, 2025, 6:00 PM | Meetup